Custom Unity Editor Tools for Synty Assets

This document outlines the development of custom Unity Editor tools designed to streamline workflows, particularly when working with modular asset packs from Synty. These tools aim to automate repetitive tasks, enhance asset management, and ultimately improve efficiency in scene creation and content generation. The project also considers the potential for commercial release on the Unity Asset Store.1. Introduction

Working with large collections of modular art assets, such as those provided by Synty Studios, often involves tedious and time-consuming tasks like sorting, placing, and generating variations of numerous small prefabs. Custom Unity Editor tools can significantly alleviate these pain points, freeing up developers to focus on creative aspects of game development. This document proposes several high-impact tool ideas and outlines the architectural considerations for building professional-grade tools suitable for both internal use and potential sale on the Unity Asset Store.2. Core Architectural Principles

To ensure robustness, maintainability, and marketability, custom editor tools will adhere to the following architectural principles:2.1 User Interface (UI): UI Toolkit

For creating intuitive and professional editor interfaces, Unity's **UI Toolkit** will be utilized over the older Immediate Mode GUI (IMGUI).

Feature	UI Toolkit	IMGUI (Immediate Mode GUI)
Authoring	Visual (UI Builder), separating visuals from logic (HTML/CSS-like)	Entirely in C# code, leading to messy and unreadable UIs for complex tools
Performance	Event-driven, redrawing only when changes occur (efficient)	Redraws the entire UI every frame (resource-intensive)
Styling	Uses USS (Unity Style Sheets) for flexible and powerful styling	Styling done line-by-line in code, hindering consistent design
Best For	Polished, complex, professional editor tools for large projects	Simple, quick debug windows or inspectors

2.2 Data Management: ScriptableObjects

Tool data, such as asset lists or configuration settings, will be stored using **ScriptableObjects**. This approach ensures data persistence and separation of concerns.

Benefits of ScriptableObjects:

- **Persistence:** Data is saved as a `.asset` file, retaining information across Unity sessions.
- **Decoupling:** Separates tool data from its logic, allowing UI windows to be closed without data loss.
- **Shareability:** Enables multiple tools or systems to reference and share the same data.

Example: `PrefabPaletteData` ScriptableObject

```
using UnityEngine;
using System.Collections.Generic;

// This attribute allows you to create instances of this object from the Assets menu
[CreateAssetMenu(fileName = "NewPrefabPalette", menuName = "My Tools/Prefab Palette")]
public class PrefabPaletteData : ScriptableObject
{
    public string paletteName;
    public List<GameObject> prefabs;
}
```

This example demonstrates how a `PrefabPaletteData` ScriptableObject can store a list of GameObjects (prefabs) for a custom palette, making the data reusable and persistent.3. Proposed Custom Editor Tools

Based on the challenges of working with modular Synty assets and the user's specific asset library (including *POLYGON - City Characters*, *POLYGON - Fantasy Characters*, *POLYGON MINI - Fantasy Characters*, *POLYGON - Dungeon Pack*, *POLYGON - Farm Pack*, etc.), the following high-value tools are proposed:3.1 Unified Asset Palette and Search Tool

This tool will serve as a central hub for discovering, organizing, and quickly accessing all Synty assets within a project.3.1.1 Purpose

To eliminate the need for manual folder navigation by providing a fast, visual, and searchable interface for the entire Synty collection.3.1.2 Data Architecture (ScriptableObjects)

- **`SyntyPackData` ScriptableObject:** Stores the pack's name and references to all its prefabs.
- **`PrefabData` Class:** A serializable class within `SyntyPackData` containing the prefab reference, its category (e.g., "Characters," "Buildings"), and its pack name.
- **Persistence:** Data is persistent and can be regenerated on demand.
- **Implementation:** A custom Editor script will automatically scan asset folders for Synty packs and generate/update `SyntyPackData` ScriptableObjects for each.

3.1.3 User Interface (UI Toolkit)

- **`EditorWindow`:** The main user interface.
- **`ToolbarSearchField`:** For real-time asset filtering.
- **`TwoPaneSplitView`:** To display a list of packs on the left and a grid of prefabs on the right.
- **`ListView`:** For the prefab grid, utilizing custom `makeItem` (UI element creation) and `bindItem` (data population) functions.
- **Asset Previews:** `AssetPreview.GetAssetPreview()` will be used to generate and display thumbnails for each prefab asynchronously.

3.2 Prefab Painter with Advanced Randomization

This tool will significantly accelerate the placement of detailed props across scenes.3.2.1 Purpose

To rapidly populate scenes with environmental details (e.g., rocks, bushes) from various Synty packs, while ensuring natural variation.3.2.2 Data and Implementation

- `PrefabBrush` **ScriptableObject:** Stores a list of prefabs and randomization settings (scale range, rotation, alignment to surface normal).
- **Scene View Interaction:** The main Editor script will implement `OnSceneGUI` to draw a brush preview and handle mouse input.
- **Placement Logic:** When painting, the tool will instantiate a random prefab from the `PrefabBrush` and apply configured randomization settings.

3.2.3 Specific to Synty Assets

- **Mixed Brushes:** Ability to combine props from different packs (e.g., "Nature Pack" rocks and "War Pack" debris).
- **Brush Generator:** Feature within the Asset Palette tool to create `PrefabBrush` ScriptableObjects by dragging selected assets.

3.3 Modular Structure Assembly Tool

This tool is essential for quickly assembling complex structures from modular kits, particularly from the Dungeon, Fantasy Kingdom, and Office packs.3.3.1 Purpose

To streamline the assembly of modular walls, floors, and roofs by automatically snapping pieces together based on defined connection points.3.3.2 Data and Implementation

- **Connection Points:** Defined on modular prefabs using empty GameObjects with specific tags or custom MonoBehaviour components.
- **Snap Logic:** The Editor script will track selected prefabs and snap new pieces to the nearest available connection point.
- **Building Generator:** For packs like Fantasy Kingdom, a ScriptableObject can define house layouts, allowing designers to place pre-assembled house variants.

3.3.3 User Interface (UI Toolkit)

- `EditorWindow`**:** Features a drag-and-drop area for modular pieces.
- **Controls:** For snapping settings and visualizing connection points.
- **"Build Mode" Button:** Activates special scene view interactions handled by `OnSceneGUI`.

3.4 Character Customization and Randomizer

This tool automates the creation of unique characters from modular character packs.3.4.1 Purpose

To quickly generate a large number of unique non-player characters (NPCs) by mixing and matching modular character parts.3.4.2 Data Architecture and Implementation

- **Character Part Identification:** System for categorizing character parts (e.g., body, head, hat, weapons).
- `CharacterPart` **ScriptableObject:** Can be used to organize and reference character parts.
- **Randomization:** A "Randomize" button will assemble new characters by randomly selecting parts from each category.
- **Variant Generation:** Option to generate new Prefab Variants of assembled characters for persistent storage.

3.4.3 User Interface (UI Toolkit)

- `EditorWindow`**:** Displays all available character parts and allows users to mix and match on a preview character.
- **"Randomize" Button:** Triggers the character assembly.

4. Preparing for Commercial Release (Unity Asset Store)

Transforming a functional tool into a commercial product requires a focus on user experience, marketing, and ongoing support.4.1 Professional Store Page

The Asset Store page is the primary marketing interface.

- **Key Art (Icon, Card, Cover):** Clean, professional, and clear communication of the tool's function.
- **Screenshots and GIFs:** High-quality visuals demonstrating the tool in action.
- **Video Trailer:** A short (1-2 minute) video showcasing the problem, the tool's solution, and key features.

4.2 Comprehensive Documentation

Documentation is mandatory for assets with code or setup requirements.

- **Format:** Included as PDF, Markdown (.md), or text file within the package.
- **Content:**
    - Installation & Setup guide.
    - Quick Start Guide for main features.
    - Detailed breakdown of all features, buttons, and settings.
    - Contact information for support.